Parallel Computing Project Report: *Wave Function Collapse*

Filippo Battisti, Simone Freschet, Francesco Bittasi

Academic Year 2025/2026

1 Introduction

Procedural Content Generation (PCG) is a fundamental challenge in computational design, requiring algorithms to produce complex, randomized, yet strictly rule-abiding structures. The Wave Function Collapse (WFC) algorithm, originally introduced by Maxim Gumin in 2016 [4], elegantly addresses this by framing procedural generation as a formal constraint satisfaction problem. Operating on principles inspired by quantum mechanics, the algorithm initializes a domain where every cell exists in a superposition of all possible states. Through an iterative process of observation and propagation, the algorithm continuously collapses the lowest-entropy cells and strictly enforces adjacency constraints until a globally valid configuration is achieved.

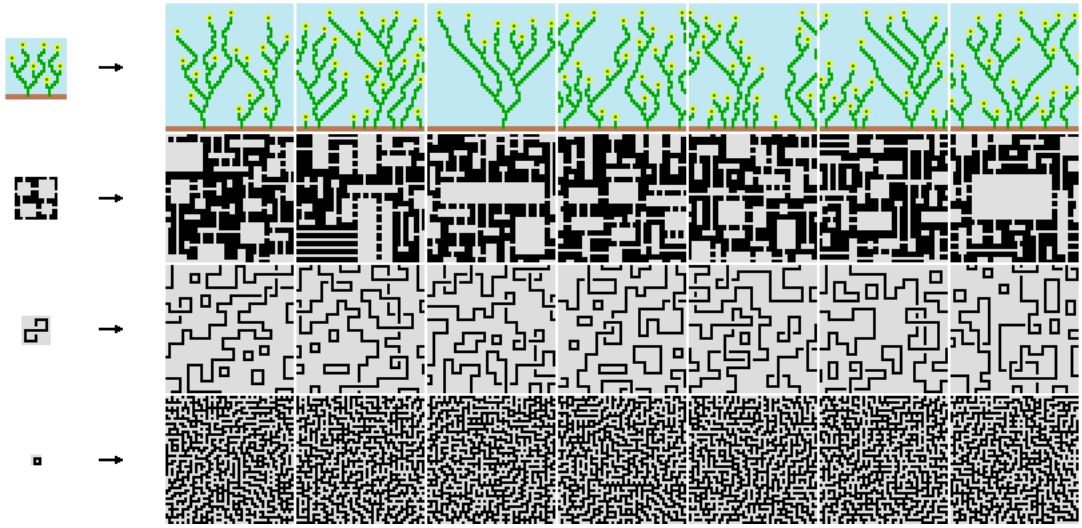


Figure 1: Examples from the Maxim Gumin Github repository.

1.1 Parallel Implementation Challenge

In recent years, numerous researchers have sought to optimize the computational performance of the WFC algorithm. Notable advancements include the implementation of graph-based data structures to extend WFC beyond rigid spatial grids [7], as well as the development of hybrid frameworks that combine constraint solving with traditional procedural noise to bypass

sequential bottlenecks [3]. However, academic literature concerning the direct, distributed parallelization of the core WFC algorithm remains remarkably sparse. This gap is primarily due to the algorithm's inherently non-local constraint propagation: a single cell collapse can trigger a cascading chain reaction of state removals that ripples unpredictably across the entire domain. This creates a highly strict data-dependency structure, where the state of any given cell relies entirely on the successful resolution of constraints originating from distant coordinates. Consequently, true parallel WFC remains a significant and open computational challenge.

To make parallelization feasible, it is strictly necessary to limit the propagation diameter. Once a hard bound on constraint propagation is guaranteed, the computation on a distributed system becomes feasible. In this work, we resolve the sequential bottleneck through a parallel design built upon the Message Passing Interface (MPI). By subdividing the generation domain into nodes and enforcing a policy of exclusive access via mask sliding, we guarantee a conflict-free execution. Crucially, to properly handle the required boundary awareness between adjacent subdomains, our implementation utilizes asynchronous halo-exchange techniques via the MPI interface. This mapping ensures synchronized, overlapping boundary resolution without deadlocks, allowing all nodes to advance in parallel while maintaining the statistical correctness of the final generation.

1.2 Report Structure

The remainder of this report is structured as follows. Section 2 details the standard sequential algorithm, establishing the baseline mechanics and strict global data dependencies. Section 3 outlines the parallel design, introducing the conflict-free mask sliding policy. Section 4 presents the topology implementation and the specific tile sets used to limit the propagation bound. Section 5 discusses the technical nuances of the MPI implementation, specifically focusing on domain decomposition, halo exchanges, and centralized data aggregation. Finally, Sections 6, 7, and 8 cover the experimental setup on the CAPRI HPC infrastructure, the execution results, and a critical discussion of the algorithm's scalability limitations and improvements.

2 Sequential Algorithm

This section outlines the standard sequential implementation of the Wave Function Collapse algorithm, framing procedural generation as a formal constraint satisfaction problem. By detailing the core observation and propagation loop alongside its computational complexities, we establish the baseline mechanics and highlight the strict global data dependencies that pose challenges for future parallelization.

2.1 Problem Definition

The WFC algorithm frames procedural generation as a constraint satisfaction problem. Let the generation domain be a 2D grid G of dimensions $\sqrt{N} \times \sqrt{N}$, containing a total of N cells.

- **Inputs:**
  - A discrete set of possible tile states or patterns, $T = \{t_1, t_2, \ldots, t_k\}$.
  - A set of adjacency constraints C, which dictate whether state t_a can be placed adjacent to t_b in a specific direction $d_{ab} \in \{\text{up, down, left, right}\}$.
  - A set of weights W, representing the frequency or probability of each state appearing.
- **State Representation:** Each cell $c_{ij} \in G$ maintains a superposition of currently valid states, denoted as $S_{ij} \subseteq T$ and its entropy $|S_{ij}|$. Initially, the state is completely unobserved, meaning $S_{ij} = T$ for all c_{ij}.

- **Outputs:** A fully collapsed grid where every cell has exactly one valid state ($|S_{ij}| = 1$), satisfying all constraints in C. If any cell's entropy reaches zero, the grid contains an unresolvable contradiction, resulting in a failure state.

2.2 Algorithm and Complexity

The standard sequential WFC algorithm loops continuously through two primary phases: Observation and Propagation.

1. **Observation:** The algorithm finds the uncollapsed cell (where $|S_{ij}| > 1$) with the lowest entropy to reduce the possibility of future contradictions (zero-entropy states). This phase is implemented with a priority queue extraction to ensure $\mathcal{O}(\log N)$ lookup time. The chosen cell is then collapsed by pseudo-randomly selecting a single state from its remaining superposition, guided by the input weights W.

2. **Propagation:** Once a cell is collapsed, its updated state restricts the valid states of its immediate neighbors. The algorithm recursively propagates these restrictions outward: if a neighbor's superposition is reduced, the propagation continues, and its subsequent neighbors are checked. This cascading chain reaction continues until the neighbor states are no longer affected.

When the whole grid has been collapsed, and thus the priority queue is empty, the execution stops.

Algorithm 1 Sequential Wave Function Collapse

Input: Grid G of dimensions $\sqrt{N} \times \sqrt{N}$, Tile set T, Constraints C and Weights W.
Output: Collapsed grid G or FAILURE
for $i, j \leftarrow 0$ to $\sqrt{N} - 1$ **do**
    $S_{ij} \leftarrow T$
end for
$Q \leftarrow G$ ▷ Add all cells to the priority queue
while Q.NOTEMPTY() **do**
    $c_{min} \leftarrow Q$.GETMIN()
    if $|S_{c_{min}}| = 0$ **then**
        return FAILURE ▷ Contradiction: A cell has 0 possible states
    end if
    $t \leftarrow$ RANDOMSTATE$(W, S_{c_{min}})$
    $S_{c_{min}} \leftarrow \{t\}$ ▷ Collapse the cell to a single state
    for $n \in$ NEIGHBORS(c_{min}) **do**
        PROPAGATE(c_{min}, n, Q, G, C)
    end for
end while
return G

function PROPAGATE(a, b, Q, G, C)
    $d_{ab} \leftarrow$ direction from a to b
    Removed $\leftarrow$ FALSE
    for $t \in S_b$ **do**
        if no state in S_a allows t based on C, d_{ab} **then**
            $S_b \leftarrow S_b \setminus \{t\}$
            Removed $\leftarrow$ TRUE
        end if
    end for
    if *Removed* **then** ▷ Neighbor modified, must propagate further
        Q.UPDATE(b) ▷ Update neighbor's entropy in queue
        for $n \in$ NEIGHBORS$(b) \setminus \{a\}$ **do**
            PROPAGATE(b, n, Q, G, C)
        end for
    end if
end function

Time Complexity Let $|T|$ be the total number of possible states. Across the entire execution of the algorithm, a state can be removed from a cell at most $N \cdot |T|$ times. Processing each removal requires evaluating the constraints against the 4 adjacent neighbors, taking $O(|T|)$ time per neighbor. Therefore, the amortized time complexity of the propagation phase across all iterations is $\mathcal{O}(N \cdot |T|^2)$. Furthermore, priority queue operations take $\mathcal{O}(\log N)$ time per update and extraction. Because a cell's priority must be updated each time its entropy drops, which can occur up to $N \cdot |T|$ times, standard implementations yield an overall time complexity of $\mathcal{O}(N \cdot |T|^2 + N \cdot |T| \log N)$.

Space Complexity The algorithm must store the superposition matrix for the entire grid, which requires $\mathcal{O}(N \cdot |T|)$ space, while the entropy priority queue requires at most $\mathcal{O}(N)$ space. Additionally, evaluating the adjacency constraints using a 3-dimensional bit-matrix (states $\times$ states $\times$ directions) requires $\mathcal{O}(|T|^2)$ space. Therefore, the overall space complexity is bounded by $\mathcal{O}(N \cdot |T| + |T|^2)$.

Data-Dependency Structure The core data dependency in WFC lies in the propagation stack. A single collapse at cell c_{ij} can trigger a cascading chain reaction of state removals that ripples across the entire grid G. This creates a highly strict, dynamic, and non-local dependency structure: the state of any given cell relies entirely on the successful resolution of constraints originating from completely unpredictable, distant coordinates. This inherently sequential, globally linked constraint propagation is exactly what makes the algorithm difficult to parallelize. As we will see in the following section, however, once a strict limitation in the propagation diameter is determined, a parallel implementation of the algorithm becomes feasible.

3 Parallel Design

Sequential WFC exhibits a strict dependency between successive collapses: each collapse modifies the superposition set of adjacent chunks, conditioning future choices. This dependency makes parallelism non-trivial to extract, as a naïve parallelization in which multiple chunks are collapsed simultaneously without coordination can produce contradictory states or violate the adjacency constraints defined by the model.

The approach described here resolves this problem through a decomposition of the domain into nodes and a policy of exclusive access to chunks via mask sliding, which guarantees by construction the absence of conflicts between concurrent collapses. The result is an algorithm in which all nodes advance in parallel synchronously, each operating in isolation on its own subdomain, with communications exclusively local to immediate geometric neighbors.

The main properties of the implementation are the following:

- **Statistical correctness:** The parallel output is statistically equivalent to the sequential one that is, it belongs to the same distribution of valid configurations while not being deterministically identical given the same seed, as the global order of collapses differs between the two formulations.
- **Communication locality:** Each node communicates exclusively with its immediate geometric neighbors. No global communications are present during the collapse phase, with the exception of barrier synchronizations between successive slides.
- **Architecture independence:** The adopted parallelism model makes no assumptions about the underlying hardware: the decomposition into nodes is applicable to distributed-memory systems via MPI, shared-memory systems via threads, and hybrid models alike.
- **Conflict-free by construction:** The geometry of the sliding guarantees that no chunk is ever covered simultaneously by two distinct masks, eliminating the need for locking mechanisms or runtime conflict resolution.

For this implementation we define the following elements:

Node: A set of 3×3 chunks whose evolution is managed by a single MPI process or thread. The node is the unit of work of the parallel system: it performs the collapse of its own chunks and manages the exchange of updates with adjacent nodes.

Mask: A 3×3 chunk window centered on the chunk currently being collapsed. It defines the set of chunks that may be influenced by the propagation originating from the current collapse, potentially extending onto chunks belonging to adjacent nodes.

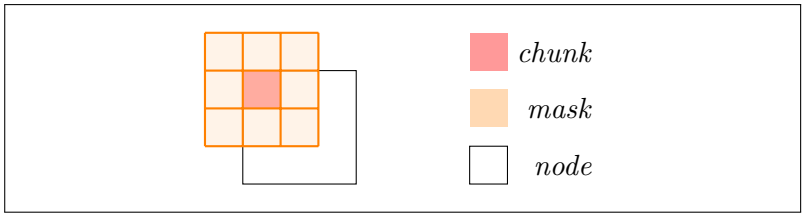


Figure 2: Visual representation of chunk, mask and node

Mask sliding and update ordering The parallelism of the algorithm is founded on the geometric property by which the masks of distinct nodes never overlap on the same chunk at the same instant. To guarantee this property, the mask slides over the 9 chunks internal to the node following a row-major ordering, visiting positions in sequence from the bottom row to the top, left to right. At each sliding step, the mask is centered on a different chunk, and the geometry of the system ensures that the current chunk is not simultaneously covered by the mask of any adjacent node, making the local collapse conflict-free.

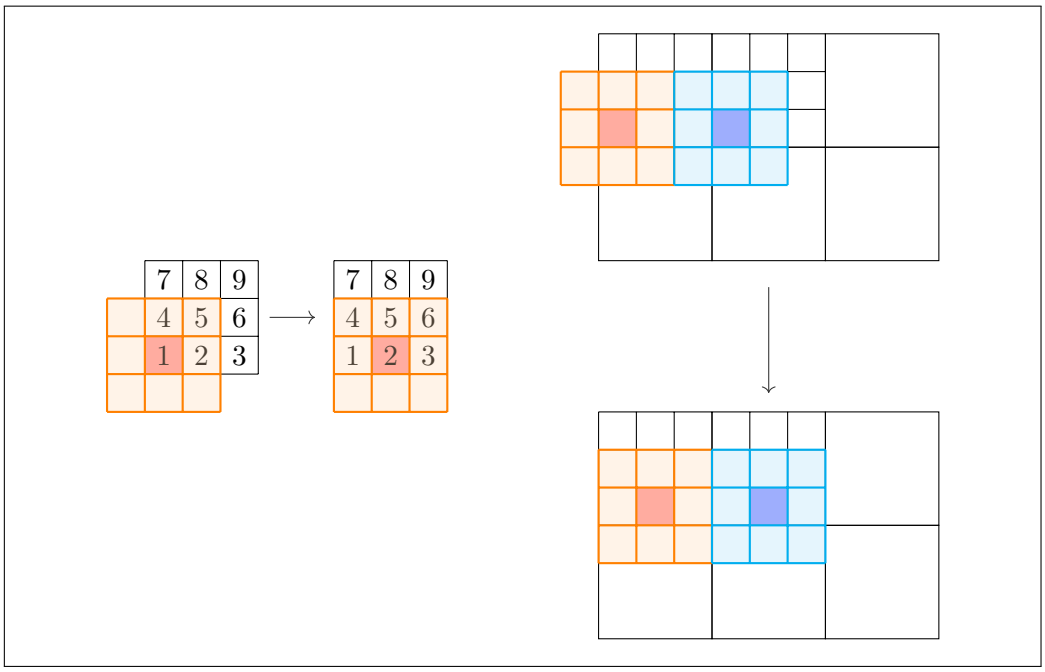


Figure 3: Visualization of the mask sliding update order

When the mask extends onto chunks belonging to a neighboring node, the owner node sends a copy of those chunks to the node that needs to update them. The latter performs the propagation, updates the received copies, and sends them back to the owner before the next sliding step. The correctness of this exchange is guaranteed by the fact that, by construction, the owner node does not cover with its own mask the temporarily transferred chunks, eliminating any access conflict.

All nodes advance synchronously: each sliding step is preceded by a global synchronization phase in which each node sends and receives updates relative to the chunks shared with its neighbors, after which it frees the memory of chunks no longer needed for the subsequent iteration. Chunks that fall beyond the boundaries of the global domain are simply ignored.

3.1 Communication Pattern

The communication pattern arising from mask sliding is local and point-to-point: each node communicates exclusively with the geometric neighbors that share a border or a vertex with it in the node grid.

Communications occur via asynchronous exchange, allowing the internal computation of the node to overlap with the transfer of boundary data. The volume of data exchanged per synchronization step is proportional to the number of chunks shared with each neighbor, the full superposition set of each chunk that is, the set of compatible patterns for each tile and the thickness of the boundary region determined by the maximum extent of propagation.

Time Complexity Let P be the number of nodes and L the side length of a chunk in tiles. Each node manages a 3×3 subdomain of chunks, and its local computation is equivalent to running sequential WFC on $9 \cdot L^2$ cells. The time complexity per node is therefore $\mathcal{O}(L^2 \cdot |T|^2 + L^2|T| \log L)$, identical to the sequential case scaled to the subdomain size.

Since all nodes advance synchronously and operate in isolation on disjoint subdomains, the P nodes execute their local computation in parallel, yielding a parallel time complexity for the computation phase of:

$$T_{comp}(N, P) = \mathcal{O}\left(\frac{N}{P} \cdot |T|^2 + \frac{N}{P}|T| \log \frac{N}{P}\right)$$

where N is the total number of cells in the domain. To this must be added the synchronization overhead T_{sync}, incurred at each of the 9 sliding steps. during which each node exchanges the superposition sets of the chunks shared with its neighbors.

$$T_{sync}(N, P) = \mathcal{O}\left(\frac{N}{P}|T|\right)$$

In conclusion, the total time complexity for the parallel algorithm is the combination of T_{comp} and T_{sync}

$$T(N, P) = T_{comp}(N, P) + T_{sync}(N, P) = \mathcal{O}\left(\frac{N}{P} \cdot |T|^2 + \frac{N}{P}|T| \log \frac{N}{P}\right)$$

Space Complexity Each node stores the superposition states of its 3×3 chunks, along with temporary copies of boundary chunks received from adjacent nodes. The local memory required to store these chunks is $18L^2 \cdot |T|$. Combined with the 3-dimensional adjacency bit-matrix, the space complexity per node is $\mathcal{O}(L^2 \cdot |T| + |T|^2)$. Consequently, the total space requirement across all nodes is $\mathcal{O}(N \cdot |T| + P|T|^2)$.

4 Topology-Aware Implementation

Transitioning the conceptual parallel design to physical hardware requires a specialized network mapping and strict algorithmic guarantees. This section details the adoption of the King's Graph topology to facilitate exchanges between all adjacent nodes. To mathematically validate that this localized communication is sufficient, we introduce $\texttt{Maze}_{16}$, a custom tileset designed specifically to bound the WFC propagation radius, ensuring that no cascading updates ever breach the isolated subdomains.

4.1 Topologies Considered

The communication pattern of the algorithm, described in the previous section, requires each node to exchange data with all its immediate geometric neighbors, including those in diagonal

directions. A standard 2D mesh with 4 neighbors is therefore insufficient: diagonal communications would require multi-hop routing through intermediate nodes, introducing additional latency and unnecessary synchronizations not required by the algorithm.

The adopted topology is the King's Graph, a two-dimensional grid in which each node is connected to its 8 immediate geometric neighbors both orthogonal and diagonal. This choice allows the algorithm's communication pattern to be mapped directly onto the physical topology, guaranteeing that every data exchange occurs in a single hop, without intermediaries.

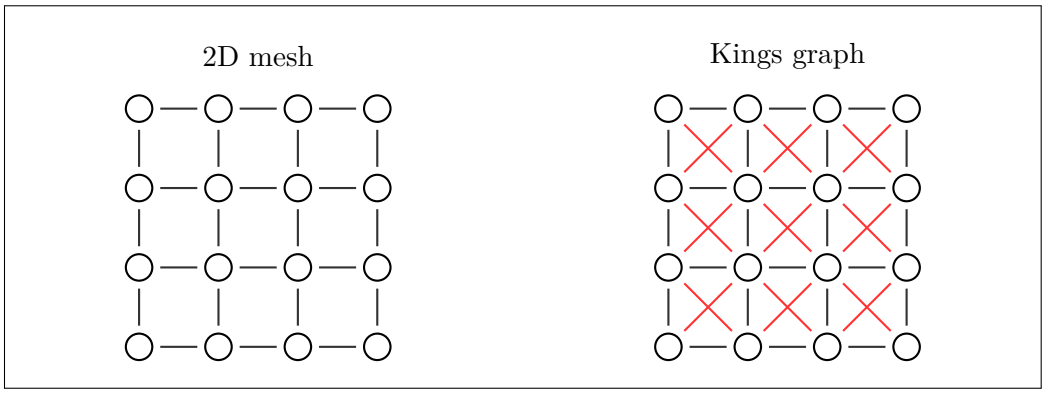


Figure 4: Representation of the 2D mesh and the Kings graph

4.2 Embedding the Communication Pattern

The embedding assigns each node with logical coordinates (i, j) in the node grid to its corresponding process, preserving two-dimensional spatial contiguity. Each process communicates directly with its 8 neighbors in the logical grid, which correspond directly to the physical neighbors in the King's Graph topology, without any routing indirection.

The correctness of this embedding, however, rests on a non-trivial assumption: that the propagation originated by any collapse is strictly confined within the chunks covered by the current mask, never spilling into the domain of a non-adjacent node. If this condition were violated, the local communication pattern described above would be insufficient, and global synchronization would be required at every step. We now establish that this condition is guaranteed by construction through the design of the tile set.

4.3 $\texttt{Maze}_{16}$

In the implementation, cells are updated via a stack that visits propagations in the following order: Up, Right, Left, Down. The algorithm begins by recursively updating cells starting from the one immediately above the collapsed cell, following the order specified above, until an update returns a cell as a superposition of all possible states at which point the cell is defined as *virgin* and no further propagation occurs.

To bound this propagation, we designed a specific tile set, hereafter referred to as $\texttt{Maze}_{16}$:

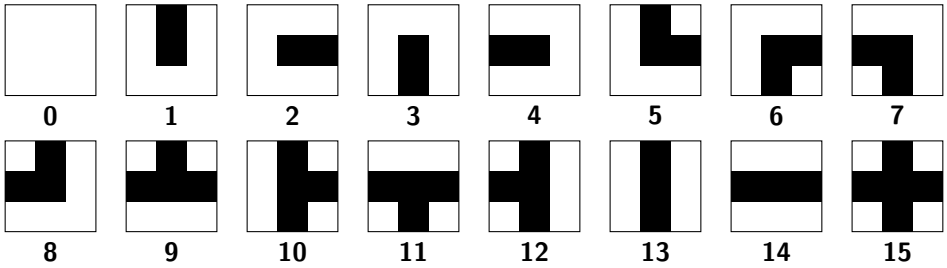


Figure 5: A representation of the $\texttt{Maze}_{16}$ tileset.

$\texttt{Maze}_{16}$ consists of 16 tiles, each identified by an index $i \in \{0, \ldots, 15\}$ and defined on a 3×3 grid of bits with fixed corners set to 0. The 16 tiles represent all possible local corridor configurations, and any valid tiling of an $N \times N$ grid produces a well-formed classical maze.

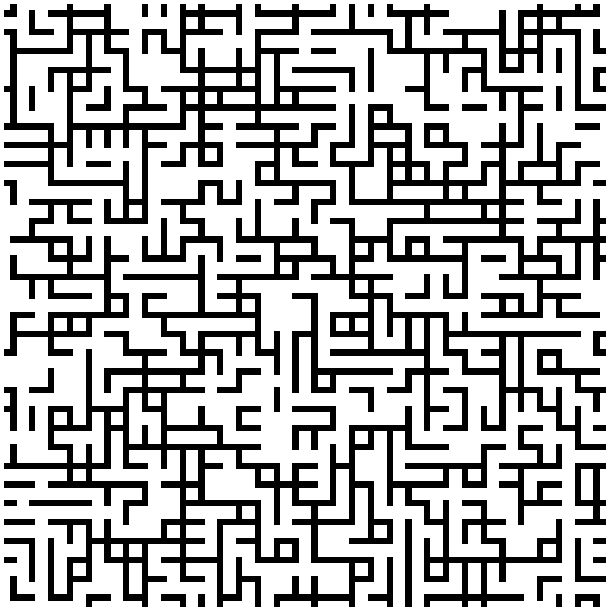


Figure 6: Example of a 32x32 maze generated by the algorithm

We define Δ_{prop} as the maximum propagation distance of an update for a given tile set. We claim:

4.3.1 $\texttt{Maze}_{16}$ Theorem on Δ_{prop}

Theorem: Given the $\texttt{Maze}_{16}$ tileset, it holds that:

$$\Delta_{prop}(\texttt{Maze}_{16}) = 1$$

Proof: Since the corners of each tile are always 0, the central pixel of each border is the only free bit, yielding exactly two possible border patterns:

- **Pattern (a):** border 010, i.e. the central pixel is 1.
- **Pattern (b):** border 000, i.e. the central pixel is 0.

We denote by $Dir(p, x)$ the update operation applied to a cell of state x in direction $Dir \in \{Up, Right, Left, Down\}$, where $p \in \{a, b\}$ indicates the border pattern exhibited by the source cell toward x. The state x is represented as a subset of $\{s_0, \ldots, s_{15}\}$, and $x = v$ (virgin) denotes the full superposition of all 16 states.

Base case single collapsed neighbor. We begin by studying propagation in the first direction visited by the algorithm: Up. Starting from a virgin cell v, the two possible updates yield:

$$Up(a, v) = \{3, 6, 7, 10, 11, 12, 13, 15\}$$
$$Up(b, v) = \{0, 1, 2, 4, 5, 8, 9, 14\}$$

We now move to the updated cell and attempt to propagate recursively. The key observation is that both resulting sets contain tiles exhibiting both border patterns toward every direction

except the one of provenance (Down). For instance, toward Up:

$$Up(a, v) \ni 3 \text{ (pattern a)},\ 10 \text{ (pattern b)}$$
$$Up(b, v) \ni 1 \text{ (pattern a)},\ 0 \text{ (pattern b)}$$

and analogously toward Right and Left. Therefore:

$$Up([a, b],\ v) = Up(a, v) \cup Up(b, v) = \texttt{Maze}_{16} = v$$

All cells adjacent to the first updated cell are necessarily returned to the virgin state and halt the propagation. By the rotational closure of $\texttt{Maze}_{16}$ which consists of 6 base tiles $\{0, 1, 5, 9, 13, 15\}$ and their rotations the same argument holds for every initial propagation direction.

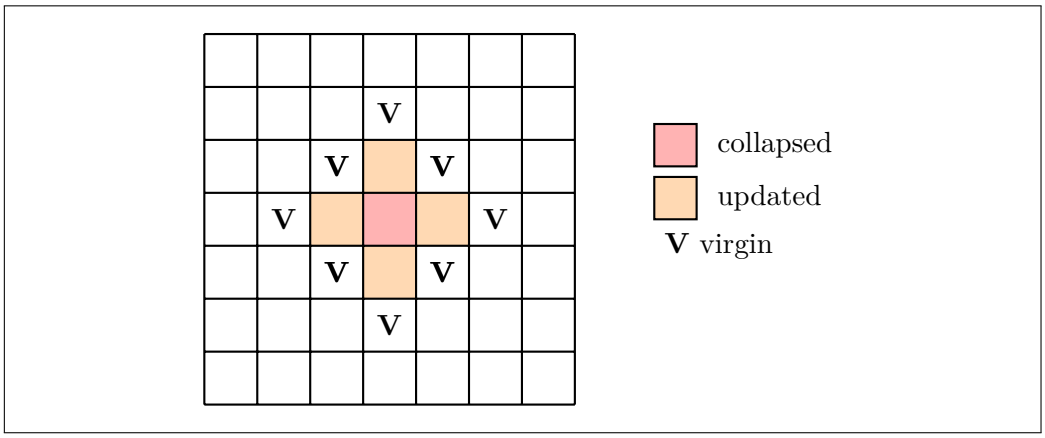


Figure 7: Propagation of $\texttt{Maze}_{16}$ without previously collapsed borders.

Extended cases multiple collapsed neighbors. In practice, successive collapses may cause a cell to be updated while more than one of its neighbors is already collapsed. We distinguish three exhaustive configurations:

Case 1: two collapsed neighbors sharing a vertex. Consider, for example, the Up and Right neighbors both collapsed. Depending on their border patterns, the possible resulting states of x are:

$$Up(a, v) \cap Right(a, v) = \{5, 9, 10, 15\}$$
$$Up(b, v) \cap Right(b, v) = \{0, 3, 4, 7\}$$
$$Up(a, v) \cap Right(b, v) = \{1, 8, 12, 13\}$$

In each case, the resulting set contains tiles exhibiting both pattern (a) and pattern (b) toward every free direction. For instance, for $\{5, 9, 10, 15\}$, denoting by Dir_p the border pattern $p \in \{a, b\}$ exhibited by a tile toward direction $Dir \in \{L, R, U, D\}$:

$$5\ (L_b, D_b), \quad 9\ (L_a, D_b), \quad 10\ (L_b, D_a), \quad 15\ (L_a, D_a)$$

All combinations of patterns toward Left and Down are represented, so both adjacent cells are returned to the virgin state. The same holds for the remaining two sub-cases.

Case 2: two collapsed neighbors not sharing a vertex. When the two collapsed neighbors are opposite (e.g. Up and Down, or Left and Right), an analogous intersection argument yields sets that still cover both patterns toward the two remaining free directions, leaving those cells virgin.

Case 3: three collapsed neighbors. This case extends Case 1: starting from a set of 4 tiles such as $\{5, 9, 10, 15\}$, a third collapsed neighbor (e.g. Down with pattern b) further restricts x to $\{5, 9\}$. This pair still exhibits both patterns toward the single remaining free direction (Left):

$$5\ (L_b), \quad 9\ (L_a)$$

and therefore leaves that cell virgin. The argument generalizes to all combinations of three collapsed neighbors by rotational symmetry.

Observation. The three cases above, together with the base case, are exhaustive with respect to propagation. The number of compatible states for a cell with k already-collapsed neighbors follows directly from the case analysis:

$$|\text{compatible states}| = 2^{4-k}$$

where $k \in \{0, 1, 2, 3, 4\}$. In all cases, as shown above, the updated cell either returns to the virgin state or is immediately collapsed, and in neither case does it propagate the update further. The entropy of adjacent cells is therefore irrelevant to the propagation bound.

Therefore, in all possible configurations, after a single update in any direction, all cells adjacent to the updated cell are necessarily virgin and do not propagate the update further, which gives $\Delta_{prop}(\texttt{Maze}_{16}) = 1$. QED

Corollary 1: This result directly validates the embedding described above: since the mask spans $L > 1$ chunks and $\Delta_{prop} = 1$, no update originating inside the mask can ever reach the boundary of an adjacent node, making the local point-to-point communication pattern both necessary and sufficient.

Corollary 2: We define a *contradiction* as a configuration in which one or more cells have entropy zero, that is, their set of compatible states is empty. From the observation above, the number of compatible states for any cell is given by 2^{4-k}, where $k \in \{0, 1, 2, 3, 4\}$ is the number of already-collapsed neighbors. Since $2^{4-k} \geq 2^0 = 1$ for all admissible values of k, the entropy of any cell is strictly bounded below by 1. Therefore, no cell can ever reach entropy zero, and no contradiction can arise during the execution of the algorithm on $\texttt{Maze}_{16}$, therefore $\texttt{Maze}_{16}$ is contradiction free.

5 MPI Implementation Notes

Transitioning the Wave Function Collapse algorithm to a parallel architecture requires overcoming the challenge of its inherently non-local constraint propagation. This section details the project's C++17 and MPI implementation, focusing on the domain decomposition, iterative boundary synchronization, and data aggregation strategies designed to achieve scalable, high-performance generation.

5.1 Language and Build

This project is implemented in C++ (specifically utilizing C++17 standard). C++ was chosen for its high execution performance, which is critical for handling computationally intensive constrained algorithms like Wave Function Collapse. Additionally, it provides low-level memory control alongside zero-cost abstractions. For instance, using structures like *bit arrays* allows for a highly efficient representation of pattern superpositions, while the language natively interfaces with the standard MPI C interface.

5.2 Notes on the Source Code

The codebase is designed with a strong emphasis on modularity, strictly decoupling the core Wave Function Collapse algorithm from specific tileset definitions and constraints. The system relies on discrete data structures: a `PWFC::RuleSet` object encapsulates the user-defined adjacency rules and pattern weights, while `PWFC::Chunk` objects manage the local spatial grids.

Because the central `PWFC::Generate()` routine operates on an abstract 3×3 matrix of these chunks, the underlying algorithm remains completely agnostic to the actual semantics or visual representation of the patterns being processed. This structural separation ensures that the library can easily accommodate custom rulesets and be integrated into external C++ projects, as demonstrated by the provided sequential and MPI-based `maze16` applications.

5.3 Domain Decomposition and Process Topology

As stated in section 3 the physical generation space is partitioned into a 2D grid of processors. The implementation thus requires the total number of processors (P) to be a perfect square.

Rather than assigning a single contiguous matrix to each node, the spatial domain within each process is further subdivided into a 3×3 local grid of chunk instances. This hierarchical subdivision is a deliberate architectural choice designed to facilitate stepwise, overlapping boundary resolution.

5.4 Iterative Overlapping and Synchronization

Because WFC requires contextual awareness of neighboring states to properly collapse each chunk, completely independent parallel generation would result in severe boundary discontinuities. The codebase solves this using a 9-round iterative execution strategy:

- **Dynamic Pivoting:** Over the course of 9 rounds, each process shifts its computational focal point, called the pivot, across its local 3×3 grid.
- **Halo Exchange:** When the 3×3 working window for a specific round extends beyond the local processor's domain, the node exchanges chunks with adjacent nodes. The synchronization pass uses `MPI_Sendrecv` to simultaneously dispatch local boundary chunks and ingest neighbor boundary chunks, effectively preventing deadlocks during the swap (The specific mathematical routing for this exchange is detailed below).
- **Implicit Boundary Handling:** The algorithm handles the global boundaries of the map by evaluating target ranks. Therefore, if a neighbor is out of bounds, the system automatically ignores communication to this out-of-bound node, allowing an unified communication logic without complex branching for edge nodes.
- **Bidirectional Updates:** After the core sequential WFC algorithm mutates the combined local and shared chunks, a second synchronization pass writes the modified neighbor states back to their original nodes. This ensures that entropy reduction and constraints propagated into halo regions are securely integrated into the global state.

5.4.1 Halo Exchange

To execute the Halo Exchange described above, the logic for routing chunks between neighboring processes relies on mapping 2D spatial offsets to absolute MPI ranks. Because each process acts as a local grid, its boundaries must seamlessly interface with the corresponding boundaries of adjacent nodes.

To determine where a chunk is sent and from where a alternative chunk is retrieved, the algorithm maps coordinates using three distinct layers:

1. **Global Node Coordinates:** Every processor calculates its own 2D position within the processor grid using its global rank r, and the grid dimension, $\sqrt{P}$:

$$x_r = r \mod \sqrt{P}, \quad y_r = \left\lfloor \frac{r}{\sqrt{P}} \right\rfloor$$

2. **The Local Window Offset:** During a given execution round, the algorithm centers its focus on a *pivot* chunk (x_p, y_p) within its local 3×3 grid. The index of the neighbor $i \in [0, 8]$ being evaluated produces a relative direction vector ranging from -1 to 1. The target destination within the global space is calculated via an offset:

$$x = x_p + (i \mod 3) - 1, \quad y = y_p + \left\lfloor \frac{i}{3} \right\rfloor - 1$$

   If either x or y falls outside the valid local range $[0, 2]$, the chunk crosses a processor boundary, triggering an external network exchange. It must be stated how the resulting values of x and y are confined in the range $[-1, 3]$.

3. **Directional Vector Evaluation:** The boundary crossing establishes a directional vector (dx_r, dy_r) indicating which neighboring processor holds the desired boundary data:

$$dx_r = \begin{cases} +1 & \text{if } x = 3 \\ -1 & \text{if } x = -1 \\ 0 & \text{otherwise} \end{cases}, \quad dy_r = \begin{cases} +1 & \text{if } y = 3 \\ -1 & \text{if } y = -1 \\ 0 & \text{otherwise} \end{cases}$$

The Send-Receive Logic A unique aspect of the design is how it maps the *receiving* node versus the *sending* node during a single `MPI_Sendrecv` call.

When a node needs to read external data from a specific neighbor, it must simultaneously send a boundary chunk to the node on the exact *opposite* side to prevent deadlocks. The target ranks are calculated by projecting the directional vectors forward and backward from the current node's coordinates:

- **Send Target Node:** located at $(x_r - dx_r, y_r - dy_r)$.
- **Receive Target Node:** located at $(x_r + dx_r, y_r + dy_r)$.

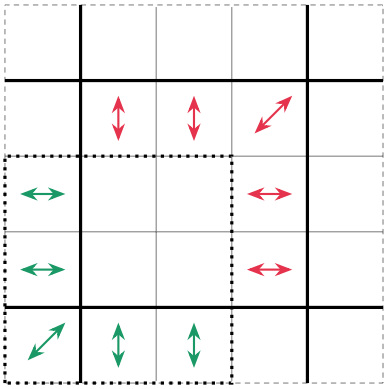
During the subsequent synchronization phase, this geometric mapping is inverted: the modified chunk data is returned to the owner processor while the updated internal data is received from the borrower node, completing the bidirectional synchronization cycle.

5.5 Centralized Aggregation and Output

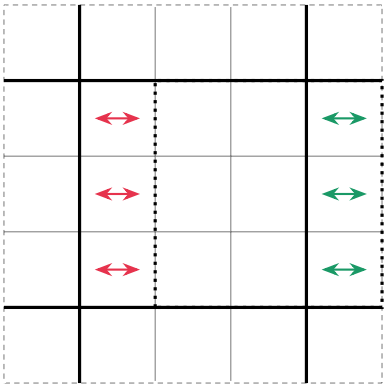
Once the 9 execution rounds conclude, the distributed map must be reconstructed.

- **Data Gathering:** The implementation leverages `MPI_Gather` iteratively (once for each of the 9 local chunk indices). All processes stream their collapsed chunk byte data to the root process (`Rank 0`).
- **Validation and Serialization:** Post-gathering, `Rank 0` translates the flat gathered buffers into a contiguous global chunk layout. A centralized validation pass acts as a safety mechanism to verify that all cells have been correctly collapsed, and that all chunk borders respect the ruleset constraints.
- **File I/O:** Finally, `Rank 0` exclusively handles serialization, translating the chunk representation into a single `PNG` image, thanks to the open source library `stb-image`[1]. This successfully decouples the parallel computation from the filesystem, avoiding concurrent write bottlenecks.

Figure 8: This picture illustrates the halo exchange during runtime.



(a) The representation of the chunk exchanges during round 1 of 9.



(b) The representation of the chunk exchanges during round 6 of 9.

6 Experimental setup

To rigorously evaluate the scalability and efficiency of our parallel Wave Function Collapse implementation, we conducted a series of benchmark experiments. This section details the experimental framework used to validate our architecture. Below, we outline the hardware specifications of the computing environment, define the parameters and methodology guiding our test cases, and describe the specific timing mechanisms used to isolate and measure computational and communication overhead.

6.0.1 CAPRI infrastructure

All measurements were conducted on the CAPRI HPC infrastructure of the University of Padua. Born as a research project to improve the HPC capabilities of the university, differentiating himself from the already available computing resources, this system features:

- 16 Intel® Xeon® Gold 6130 CPU (16 cores @ 2.10GHz)
- 6 TB DDR4 RAM
- 2 NVIDIA Tesla P100 GPU (3584 Cuda® Cores, 16 GB HBM2)

Without considering the GPU's (not used for this project), the CAPRI infrastructure has a theoretical peak performance (FMA single precision) of about 31 TFLOPS.

6.0.2 Nodes mapping

The CAPRI infrastructure features an high level abstraction that allows our application to see only one big system with 16 CPU's and 6TB of RAM, considering all CPU's fully interconnected.

The execution of our jobs on the the machine was realized via a SLURM scheduler. For this reason we weren't able to strictly choose the processors and communications used, so we expect some noise from the mapping and the other jobs running on the machine.

6.1 Methodology

To observe the average performance of the program, we executed 5 runs with different random seeds (being the running time dependent on randomness) for each configuration of

- **number of processes P**

- **size of the problem L**: the lenght of the side of the image

The same 5 seeds were used for each configuration, in this way the problem complexity was proportional to each other and thus the measumements were true indicators of the scalability of the algorithm.

Parameter	Values
Problem size L	$\{1000, 2000, 3000, 4000, 5000, 6000\}$
MPI processes P	$\{1, 4, 9, 16, 25, 36, 49, 64\}$
Seeds S	$\{123, 456, 789, 1123, 1456\}$

Table 1: Experimental parameter space

Being the jobs queued with respect to the amount of resources required, we scheduled one job for each P. Each job ran all of the 5 runs for each configuration of L.

Having executed all the jobs, python scripts were used to gather the data from the logs and conduct the analysis reported in the next section.

6.1.1 Measurements

While for the serial algorithm, the computation wall clock time was measured via the `clock_gettime()` function, for the MPI implementation we used the integrated function `MPI_Wtime()`: this may cause some differences in the observations made later.

In both cases the rendering time of the image on the output file as been kept out of the measurement: we only observe the total computation time of the algorithm and, in the case of the MPI implementation, we track the time spent in the communications. The two communication times are obtained measuring the time spent in functions `MPI_Sendrecv()` and `MPI_Gather()` for each node and then taking the maximum for each run.

6.1.2 Notes

While using a shared cluster with SLURM made the process of putting into execution the different tests very easy, the fact that it is a shared platform where big projects are constantly running made it impossible for us to test bigger configurations. At the time of writing, jobs with more than 64 processes have not yet been executed.

Because of this it's impossible for us to have a complete insight on how the system scales, but we are still able to conduct some useful analysis on the lower "resources-hungry" tests and make some assumptions on how it would perform.

7 Results

In this section we report all the results and statistics observed by the measurements taken described in the previous section. While some description will be available, a true interpretation of the data will be provided in the next section.

7.1 Execution Time

Comparing the execution time of the serial version of the program with respect to the MPI implementation with 1 single process (as reported in Figure 9), shows us that there are some relevant differences in the two methodologies. While this can be traced back to the difference in how time measurements were taken, we know that the two implementations share some important differences in how the computation is performed and thus this result can be expected.

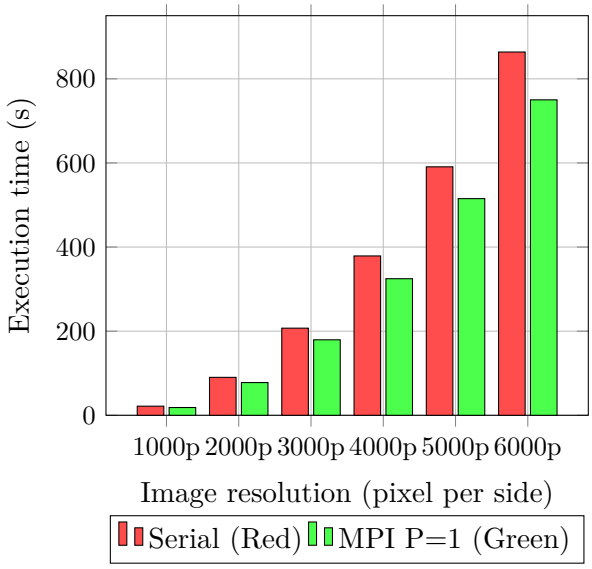


Figure 9: Analysis of execution time of the serial WFC algorithm with respect to the MPI implementation with P=1.

And for a quick insight, we observe in Figure 10 that the program is actually able to benefit a lot of the parallelization.

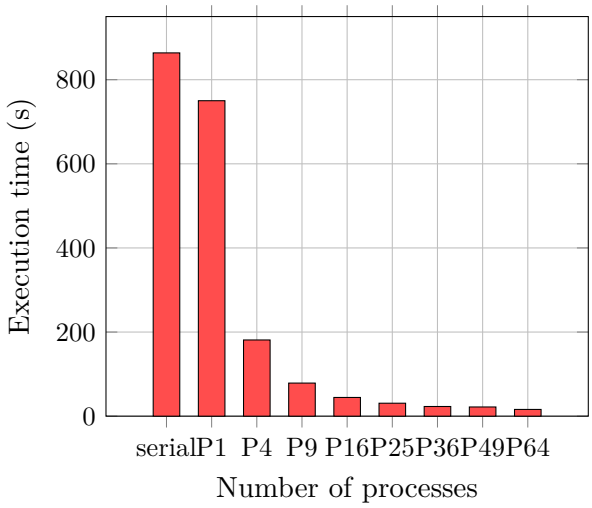


Figure 10: Execution times for L=6000 with different number of processors.

7.2 Speedup

Because of the differences observed in the previous paragraph, we analyze the speedup of the program as the **Relative Speedup** of the MPI program with P processes with respect to the execution of itself with one single process:

$$\text{Speedup relativo: } S(p) = \frac{T_{parallel}(1)}{T_{parallel}(p)}$$

Here the average speedup along the different problem size L is reported in figure Figure 11 for each configuration of P processes:

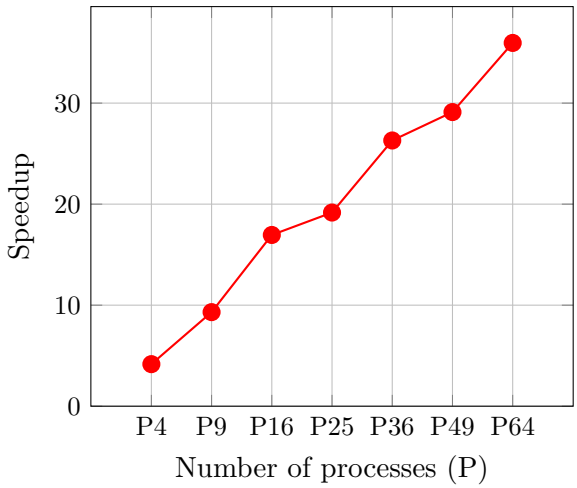


Figure 11: Average speedup for the different number of processors.

7.3 Computation vs. Communication

In Figure 12 we observe the times required by the communication procedures: the send and receive used in the different steps of the algorithm (Synch procedure) and the final gather to reconstruct the image (Gather procedure).

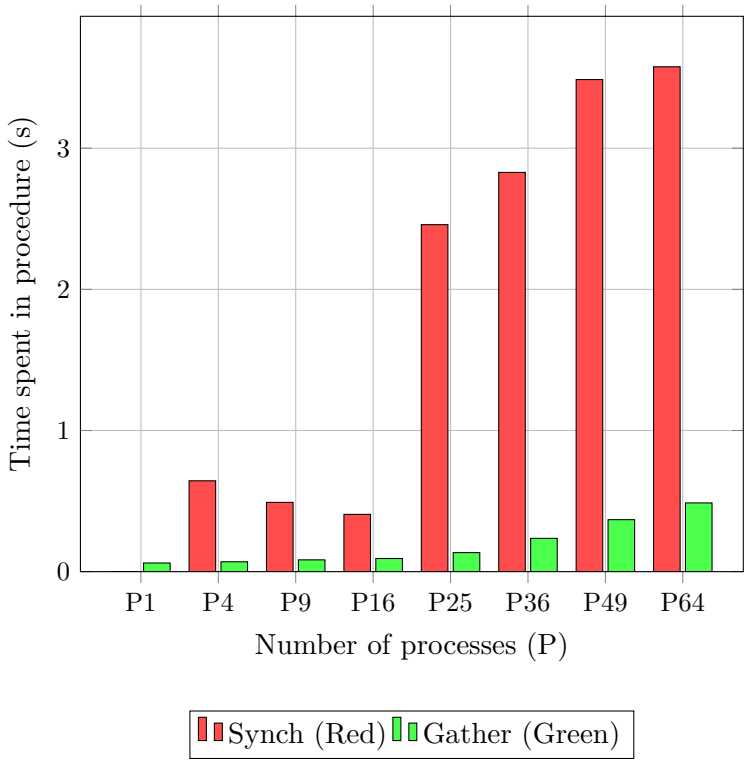


Figure 12: Average synchronization times for the different number of processors.

7.4 Strong Scaling Efficiency

With the analysis of the Strong Scaling statistic, we analyze the benefit of using more processors with problems of the same complexity: we are analyzing how efficiently we are making use of the processes.

$$E(p) = \frac{S(p)}{p} = \frac{T_{parallel}(1)}{p * T_{parallel}(p)}$$

The mean of such statistic for each configuration of P i reported in Figure 13.

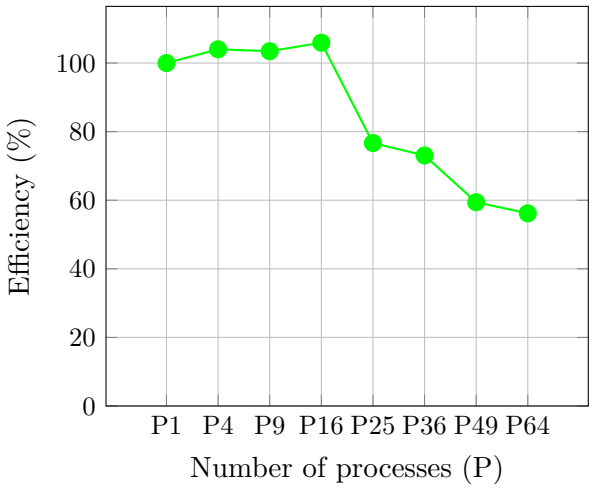


Figure 13: Analysis of the average Strong Scaling achieved with different number of processors (higher is better).

8 Discussion

With the experimental data established, we now turn to a critical evaluation of the parallel architecture. This discussion interprets the observed scaling efficiency and hardware-bound bottlenecks. Furthermore, it addresses the algorithmic and theoretical limitations of our current tileset design, and outlines actionable strategies for future optimization, specifically targeting the reduction of MPI communication overhead.

8.1 Comment on the Results

Looking at the obtained results, we observe that we achieve an optimal speedup as it seems to go develop linearly with the number of processes. Taking a closer look, even looking at the efficiency statistic, we observe that actually up to 16 processes the algorithm scales super-linearly: this can be explained by the fact that the processes are running on shared local memory and cache and this brings serious advantages.

Once the number of processes cannot be executed on a single socket, as the CAPRI infrastructure only contains cpu's with 16 cores, we observe slowdowns provoked by the time spent in the communication procedures: we see that from P16 to P25 the time pent in synch goes up by about 2 seconds.

This slowdown can be directly observed in the Efficiency study: from P25 the efficiency of our usage of the processors goes down since they spend a lot of time just waiting for the communications. Even with this slowdown the implementations remains efficient in the usage of the different processes since it stops at arount 60% of usage.

8.2 Further Notes on $\texttt{Maze}_{16}$

In its current formulation, $\texttt{Maze}_{16}$ does not guarantee the global connectivity of the generated maze: the WFC algorithm may produce multiple disconnected corridor components that are locally valid with respect to the tile constraints but do not form a single traversable structure.

Enforcing global connectivity requires a two-phase post-processing pipeline, which represents the standard approach in procedural content generation for maze-like environments [5]:

Flood fill: Once generation is complete, a flood fill traversal is performed from an arbitrary passable cell. All cells reachable from the starting point are marked as belonging to the main connected component; all remaining passable cells are identified as isolated regions. This step runs in $\mathcal{O}(N)$ time and provides a complete map of the connectivity structure of the generated maze.

Selective regeneration: Isolated regions identified by the flood fill are discarded and their cells reset to the virgin state. The WFC algorithm is then re-run locally on these regions, with the boundary constraints inherited from the surrounding already-collapsed cells, until a fully connected configuration is produced.

While this pipeline may appear inefficient due to the potential for repeated local regenerations, it is consistent with the state of the art in constraint-based procedural content generation, where post-processing validation passes are standard practice to enforce global structural properties that lie outside the scope of local tile constraints.

The connectivity guarantee was deliberately excluded from the present implementation: the post-processing pipeline does not interact with the parallel design in any meaningful way, and no parallel formulation of the connectivity enforcement step was identified that would not introduce sufficient synchronization overhead to degrade performance below the sequential baseline. Although some interesting ideas came up during discussions.

8.3 Tileset Generalization

The implementation is designed to accommodate future extensions to additional tile sets. The central requirement for any such extension is that the chosen tile set must satisfy $\Delta_{prop} < L$, so that propagation remains confined within the mask boundary. While this property is amenable to direct proof for structured sets such as $\texttt{Maze}_{16}$, no general decision procedure exists for establishing Δ_{prop} for an arbitrary tile set.

This undecidability is a consequence of a deeper result in the theory of Wang tiles. Wang conjectured in 1961 that any finite set of tiles capable of tiling the plane could do so periodically [8]. His student Robert Berger refuted this conjecture in 1966 by proving the undecidability of the domino problem that is, no algorithm can determine in general whether a given finite tile set admits a valid tiling of the plane constructing as a corollary the first known aperiodic tile set [2]. The minimal such set was subsequently identified by Jeandel and Rao, who proved that no aperiodic set of Wang tiles exists with fewer than 11 tiles or fewer than 4 colors (where colors are the different types of border connections) [6].

As a consequence, no graph-theoretic or structural analysis of an arbitrary tile set can decide whether it satisfies the propagation bound required by the parallel algorithm. The most practical approach for validating a candidate tile set remains probabilistic: a Monte Carlo analysis over a representative sample of configurations provides an empirical estimate of Δ_{prop}, sufficient for practical purposes even in the absence of a formal proof.

8.4 Optimizing Data Transfer via Spiral Trajectory and Caching

Currently, the 9-round mask sliding mechanism evaluates the 3×3 local grid in a standard row-major order. While geometrically conflict-free, this approach introduces discontinuous jumps in the focal pivot for each round with index multiple of 3.

This overhead can be mitigated by adopting a continuous, spiral-like traversal path together with chunk caching.



(a) Row Major (b) Spiral

Figure 14: The comparison between the current (a) and future (b) round path.

Under this configuration, the spatial offset between the pivot at round k and round $k+1$ is strictly bounded to a Chebyshev distance of 1. If we represent the set of chunks covered by the mask at step k as M_k, a continuous trajectory maximizes the spatial intersection between consecutive iterations:

$$|M_k \cap M_{k-1}| = 6$$

If we introduce a local memory cache for incoming MPI halo data, the node no longer needs to query the full boundary region. Instead, it only needs to fetch the newly exposed frontier:

$$\Delta M_k = |M_k \setminus M_{k-1}| \leq 3$$

By sliding a cached boundary window, the system requests at most 3 new chunks per iteration. This structural optimization would significantly reduce the overall volume of asynchronous `MPI_Sendrecv` data transfers, directly addressing the communication and gathering bottlenecks observed in the strong scaling analysis.

9 Conclusions

Developing this project and authoring the accompanying software has been a highly rewarding endeavor, demanding extensive research and dedication. We are deeply satisfied with the final outcomes. Our sole regret stems from the necessity to omit a few sections originally prepared during the development phase, specifically:

- **Further properties of** $Maze_{16}$
- **Extended analysis on Tile Theory**
- **Insights into the parallel development of the post-pipelining phase** mentioned in the discussion.
- **Further performance analysis** regarding the bottlenecks imposed by CAPRI and job scheduling.

In particular, the parallel properties were excluded as they would have significantly burdened the narrative without introducing substantial actionable insights.

Regarding the software performance, we are exceptionally pleased to have successfully achieved meaningful parallelization on an algorithm that is notoriously difficult to manage concurrently. The performance gains are highly encouraging, notably yielding a superlinear speedup on a single machine. While the extra-socket synchronization overhead was expected, it becomes negligible relative to the generation time as the chunk size increases.

In conclusion, this project delivers a highly modular framework that generates coherent procedural outputs when paired with any user-defined tileset that respects the necessary propagation constraints.

9.1 Declaration on Generative AI

During the preparation of this work, the authors utilized AI-based tools exclusively to assist in the drafting, refining, and formatting of the manuscript's text, ensuring that all core concepts and scientific contributions remain original. Additionally, AI technologies were employed to generate local automation scripts for experimental testing, as well as for the subsequent gathering and organization of the resulting data. Following the generation or editing of any content, the authors reviewed, curated, and edited the material, and take full responsibility for the accuracy and integrity of the final paper.

9.2 Examples of Generated Images

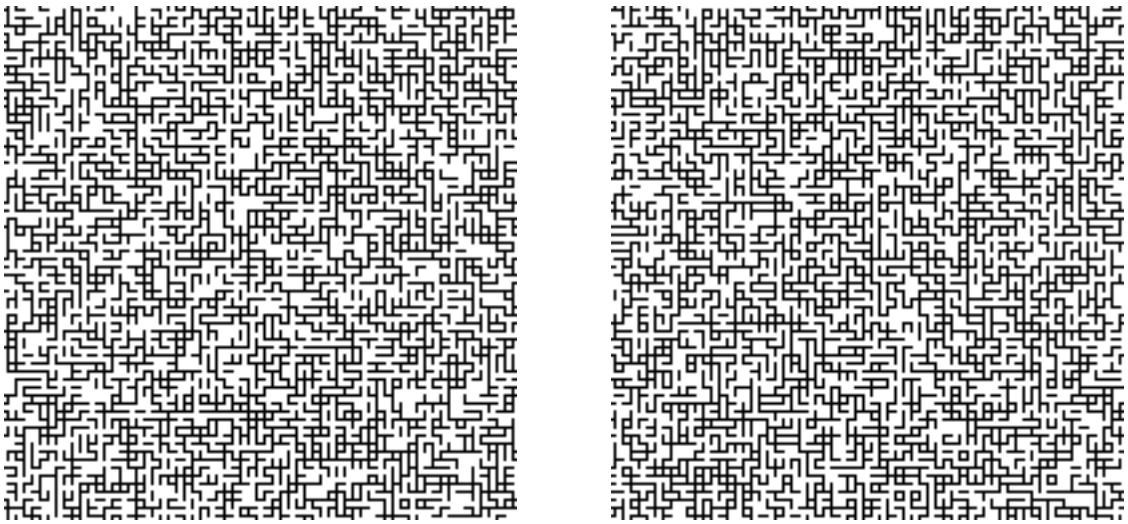


Figure 15: Mazes generated on grids 64x64.

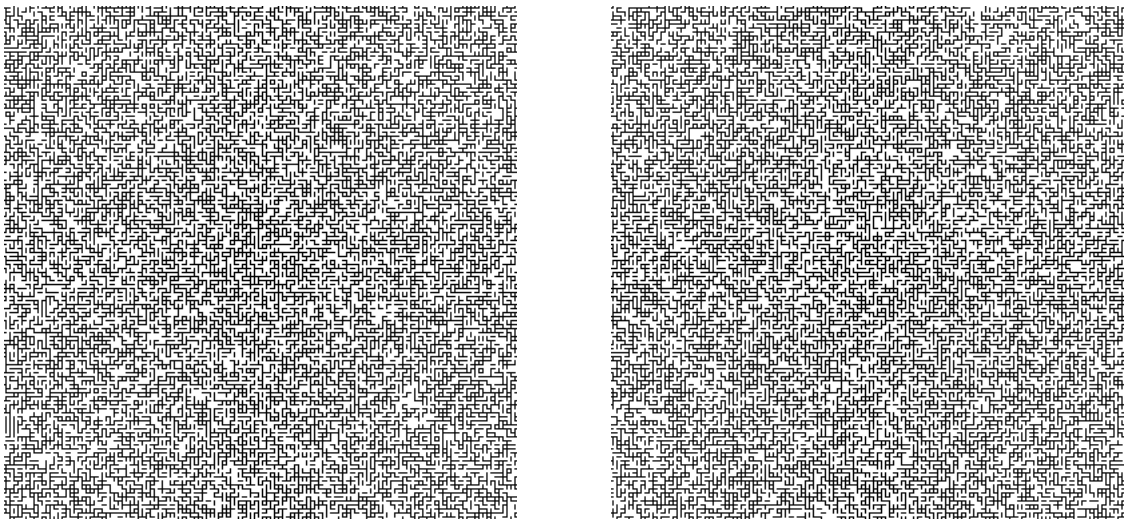


Figure 16: Mazes generated on grids 128x128.

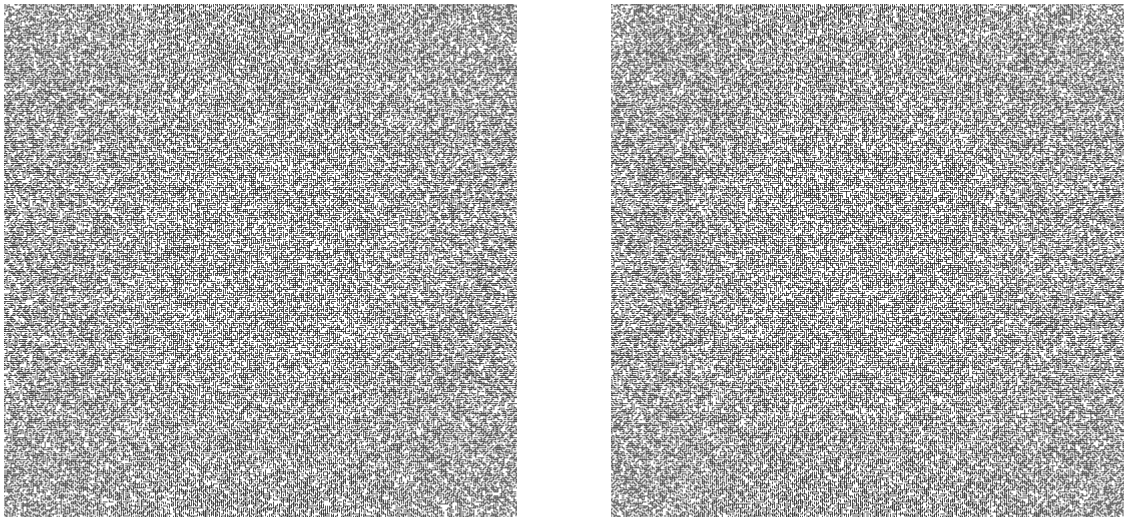


Figure 17: Mazes generated on grids 256x256.

References

[1] Sean Barret. *STB Image*. `https://github.com/nothings/stb`.

[2] Robert Berger. "The Undecidability of the Domino Problem". In: *Memoirs of the American Mathematical Society* 66 (1966), pp. 1–72. DOI: `10.1090/MEMO/0066`.

[3] Osman Büyükşar, Doğan Yıldız, and Sercan Demirci. "Enhancing wave function collapse algorithm for procedural map generation problem". In: *Niğde Ömer Halisdemir Üniversitesi Mühendislik Bilimleri Dergisi* 13.3 (2024), pp. 806–814.

[4] Maxim Gumin. *WaveFunctionCollapse*. `https://github.com/mxgmn/WaveFunctionCollapse`. 2016.

[5] Jiaping He et al. "Procedural Content Generation for Room Maze". In: *Proceedings of JIMEC*. Atlantis Press, 2018. URL: `https://www.atlantis-press.com/proceedings/jimec-18/55911409`.

[6] Emmanuel Jeandel and Michaël Rao. "An aperiodic set of 11 Wang tiles". In: *arXiv preprint arXiv:1506.06492* (2015).

[7] Hwanhee Kim et al. "Automatic generation of game content using a graph-based wave function collapse algorithm". In: *2019 IEEE conference on games (CoG)*. IEEE. 2019, pp. 1–4.

[8] Hao Wang. "Proving Theorems by Pattern Recognition II". In: *Bell System Technical Journal* 40.1 (1961), pp. 1–41. DOI: `10.1002/j.1538-7305.1961.tb03975.x`.